

源码结构

libterae 在 src/、include/ 和 platform/ 中怎样分布

Emilio Schininà，自动化工程硕士
版本 0.2
2026 年 11 月 8 日

版权所有 © 2026 Emilio Schininà, 自动化工程硕士。libterae 以 GNU 通用公共许可证第 3 版或任何更高版本, 或以商业许可证发布。

目录

1 三个文件夹 **1**
1.1 每个文件夹放什么 1
1.2 各层 2
1.3 什么决定了文件夹的形状 3

2 头文件 **5**
2.1 名字 5
2.2 四种头文件 6
2.3 定义 6
2.4 程序包含什么 8

3 协议代码 **11**
3.1 对每个文件都成立的 11
3.2 按层排列的文件 11
3.3 服务器和客户端分在几个文件里 14

4 内存函数 **15**
4.1 各文件夹 15
4.2 一个文件夹里有什么 16
4.3 `PORT=auto` 怎样选择 17

5 平台 **19**
5.1 两个层次 19
5.2 顶层的文件 20
5.3 各系统的文件夹 22
5.4 Makefile 挑选文件 23

6 去哪里找 **25**

1

三个文件夹

库由三个 C 代码文件夹组成：include/、src/ 和 platform/。本文档讲它们怎样组织：每个文件夹是做什么的，它的文件怎样命名，每个文件属于库的哪一层，以及库的某一部分该去哪里找。它只讲到文件夹和文件这一层。一个文件具体做什么，写在它开头的那段注释里；库怎样构建，见《构建 libterae》。

本文所述的文件夹以 1.0.5 版为准。

1.1 每个文件夹放什么

表 1.1 三个文件夹

文件夹	放什么	编译进
include/	86 个头文件：程序所读的接口，三个版本的类型和常量，所保存的会话状态的格式，以及库要向系统索取的东西。	不编译进任何库：由程序和另外两个文件夹读取。
src/	36 个源文件：协议、服务器接口和客户端接口，让它们的会话状态在重启后仍在的部分，以及库的版本，不含系统调用、分配器和 C 库。	libterae.a，以及 make shared 的共享库
src/port/	memcpy、memmove、memset 和 memcmp，供没有 C 库的构建使用：每种处理器一个文件夹，另有一个用 C 写的，适用于任何处理器。	libterae_port.a，当 PORT 指定了文件夹时
platform/	52 个文件：套接字、等待套接字、时钟、随机字节、TLS，以及文件或闪存中的存储区。在每个系统上都一样的东西在文件夹的顶层，某个系统按自己的方式做的，在它自己的文件夹里。	libterae.a，除非 OS=none；以及共享库

source-structure-zh.tex Page 2 November 8, 2026 13:46

除这三个文件夹外，一份克隆里还有 Makefile，它挑选一次构建所用的文件；还有 README.md 与 README.zh-CN.md（图片放在 assets/）、各许可证，以及 docs/，它以 PDF 存放文档，每种语言一个文件夹。

1.2 各层

图 1.1 把库的各部分分成若干层。每一部分只调用本层及其下各层的部分，如左边的箭头所示。右边的花括号标出每一层所在的文件夹。虚线表示库以外的东西。

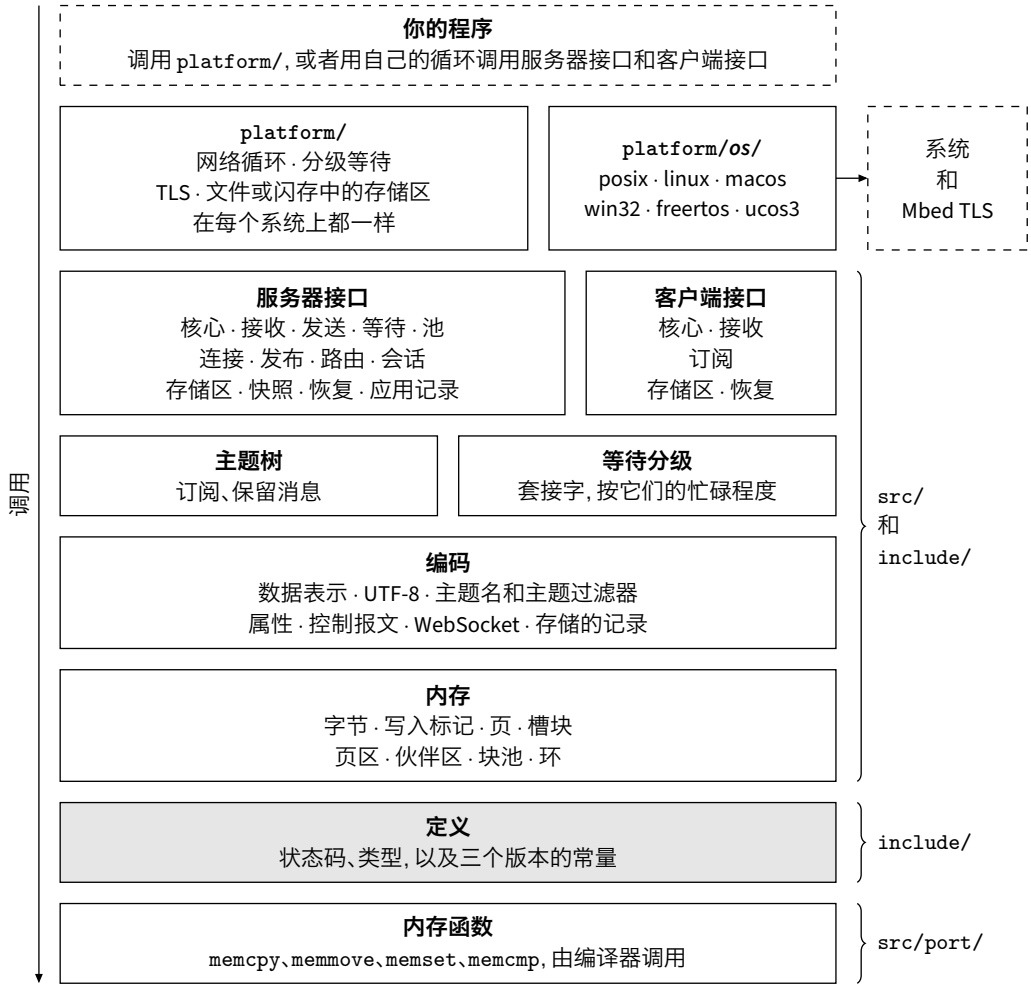


图 1.1 库的各层，以及各层所在的文件夹

source-structure-zh.tex Page 3 November 8, 2026 13:46

- **你的程序**。在库以外。它让 platform/ 运行网络循环，用 mqtt_net_server_run 或 mqtt_net_client_step；或者用 OS=none 构建，在自己的循环里自行调用服务器接口和客户端接口。
- **platform/**。库针对某个系统的特化。它运行服务器或客户端的网络循环：等待套接字，读写它们，计时，处理 TLS，在文件或闪存中保存会话状态的存储区，并把字节交给服务器接口和客户端接口。文件夹顶层的文件在每个系统上都一样。每个系统一个文件夹，即图中的 platform/*os*/，负责那个系统按自己的方式做的事，也是库里唯一调用系统的代码（第 5 章）。
- **服务器接口和客户端接口**。库的 API：程序或 platform/ 为实现一个服务器或一个客户端而调用的东西。它们收进字节，交出字节；协议留给服务器或客户端自行决定的每一件事，都由它们决定。图中它们下面的词对应它们的文件名，如“路由”对应 base_server_route.c（第 3 章）。
- **主题树和等待分级**。供上面各层使用的两个结构：树存放服务器的订阅和保留消息，等待分级把 platform/ 的套接字按忙碌程度分级，使每一次等待只需查看其中少数几个。
- **编码**。规范中的数据表示、UTF-8 和主题的检查、MQTT 5.0 的属性、控制报文的解码与编码、WebSocket 帧，以及存储区的记录和对它们的检查。
- **内存**。不用 C 库的复制与比较、说明内存是否已被写入的标记，以及把程序给的内存分配出去的各种结构：页、槽块、两个区、一个块池，以及存放传输中字节的环。
- **定义**。只有类型和常量的头文件，没有源文件，上面的每一层都要用到（第 2 章）。
- **内存函数**。即使在没有 C 库的构建里编译器也可能调用的四个函数，来自 src/port/（第 4 章）。有 C 库时，由 C 库提供它们。

1.3 什么决定了文件夹的形状

有几条规则对三个文件夹都成立，它们解释了这些文件夹的大部分布局：

- **扁平**。src/ 和 include/ 没有子文件夹，src/port/ 的各文件夹除外。platform/ 每个系统一个文件夹，没有更深的层次。
- **按版本命名**。以 base_ 开头的名字属于三个版本共有的东西。以 v31_、v311_ 或 v5_ 开头的只属于 MQTT 3.1、3.1.1 或 5.0。platform/ 的文件以 platform_ 开头，或以其文件夹的名字开头，如 win32_socket.c。
- **一个模块，一个头文件**。一个模块是 src/ 的一个源文件，加上 include/ 里同名的、声明它提供什么的头文件：base_pool.c 和 base_pool.h。服务器和客户端各分散

source-structure-zh.tex Page 4 November 8, 2026 13:46

在几个源文件里，但各自只有一个头文件（第 3.3 节）。

- **src/ 里没有来自系统的东西**。src/ 不调用它以外的任何东西。凡是各系统不同的，都在 platform/ 里，藏在 base_platform.h、base_net.h 和 base_tls.h 这几个头文件后面。
- **没有 #if 指名某个系统**。platform/ 的顶层不包含任何系统的头文件，每个系统的文件夹只用那个系统的文档所说明的东西。一次构建取某个系统的文件，是因为 Makefile 为这次构建的 OS 列出了它们，而不是因为代码里的某个条件选中了它们。
- **别处的代码只在 src/port/ 里**。src/、include/ 和 platform/ 的每个文件都是 libterae 自己的，src/port/ 的汇编除外：那里每个文件夹的 NOTICE 说明它的文件从哪来、按什么许可证。
- **每个文件都说明自己是什么**。文件以许可证声明开头，接着一段注释，说明这个文件是做什么的；凡是依照规范的，都引用规范的节和页。

2

头文件

include/ 的 86 个头文件是程序所读的，也是 src/ 和 platform/ 所共用的。它们都在同一个文件夹里，所以程序只需要 -Iinclude。除了 <stdint.h>、<stdbool.h> 和 <stddef.h>，它们不包含 C 库的任何东西。

2.1 名字

一个头文件的名字说明它服务于哪些版本，并通过它的前缀说明它的标识符以什么开头：

表 2.1 头文件的前缀

前缀	标识符	用途
base_，42 个头文件	MQTT_ 和 mqtt_	三个版本共有的东西。
v31_，9 个头文件	MQTT31_	只属于 MQTT 3.1。
v311_，14 个头文件	MQTT311_	只属于 MQTT 3.1.1。
v5_，21 个头文件	MQTT5_ 和 mqtt5_	只属于 MQTT 5.0。

名字的三种结尾各表示一类头文件：

- **_t.h**。一个类型，别无其它，这样头文件可以共用它而不必包含更多东西：base_binary_t.h、base_payload_t.h、base_run_t.h、base_utf8_t.h 和 v5_pair_t.h。
- **_check.h**。一些函数，检查去掉这个结尾后的那个头文件所定义的东西：base_utf8_check.h 检查 UTF-8 编码字符串，其限制在 base_utf8.h 里；base_topic_check.h 检查主题，其分隔符和通配符在 base_topic.h 里。
- **_internal.h**。一个模块的源文件共用的东西，不属于库的接口。base_store_internal.h 共用的范围更广：服务器和客户端的存储区，以及 base_store.c。

source-structure-zh.tex Page 6 November 8, 2026 13:46

2.2 四种头文件

表 2.2 头文件的种类

种类	头文件	数目
src/ 里一个模块的接口	base_string.h、base_bool.h、base_page.h、base_slab.h、base_page_arena.h、base_arena.h、base_pool.h、base_ring.h、base_wire.h、base_utf8_check.h、base_topic_check.h、v5_properties.h、base_packet.h、base_websocket.h、base_tree.h、base_tier.h、base_store.h、base_server.h、base_client.h、base_library.h	20
由一个模块的源文件共用, 不属于接口	base_server_internal.h、 base_client_internal.h、base_store_internal.h	3
由 platform/ 实现	base_platform.h, 库要向系统索取的东西; base_net.h, 在真实连接上运行服务器或客户端; base_tls.h, TLS	3
定义: 类型和常量, 没有源文件	表 2.3 的头文件	60

2.3 定义

大多数头文件定义规范里的类型和常量，别无其它。三个版本共有的，和某一个版本增加的，在前缀之后同名：base_type.h 有三个版本都有的控制报文类型，v5_type.h 有只有 MQTT 5.0 才有的那一个。表 2.3 列出前缀之后的名字，并标出每个名字在哪些前缀下有头文件。type 那一行有四个记号，代表 base_type.h、v31_type.h、v311_type.h 和 v5_type.h。

source-structure-zh.tex Page 7 November 8, 2026 13:46

表 2.3 只含定义的头文件

名字	定义什么	base_	v31_	v311_	v5_
库					
status	一次库调用的结果	•			
version	协议版本	•			
run_t	内存里连续的一段字节	•			
数据表示					
binary_t	二进制数据	•			
utf8_t	UTF-8 编码字符串	•			
utf8	它的限制；它可以含有的码点	•		•	•
pair_t	UTF-8 字符串对				•
vbi	可变字节整数				•
固定头					
fixed	它的布局	•			
type	控制报文类型	•	•	•	•
flags	PUBLISH 的标志；其它报文的标志	•	•	•	•
remaining	剩余长度；大小固定的报文的剩余长度	•	•	•	•
连接					
protocol	协议名和级别		•	•	•
connect	连接标志；某个版本的 CONNECT 所含的内容	•	•	•	•
keep_alive	保活	•			
client_id	客户端标识符：它的长度；它的字符	•		•	•
connack	连接确认标志			•	•
rc	返回码，以及 MQTT 5.0 的原因码		•	•	•
session	会话过期间隔				•
port	TCP 端口			•	•

续页

source-structure-zh.tex Page 8 November 8, 2026 13:46

表 2.3 只含定义的头文件(续)

名字	定义什么	base_	v31_	v311_	v5_
websocket	WebSocket 子协议			•	•
发布与订阅					
qos	服务质量等级	•			
packet_id	报文标识符	•			
payload_t	PUBLISH 负载	•			
payload	负载格式指示				•
topic	分隔符和通配符; MQTT 3.1 里主题名的长度; 以 $ 开头的主题	•	•	•	•
share	共享订阅的前缀				•
subscribe	请求的服务质量; MQTT 5.0 的订阅选项		•	•	•
suback	SUBACK 返回码		•	•	
属性					
prop	属性标识符				•
prop_value	属性未出现时的取值, 以及它们的取值范围				•

2.4 程序包含什么

- **base_server.h 或 base_client.h**。服务器接口或客户端接口。它们各自包含自己的类型所需要的头文件。
- **base_net.h**。让 platform/ 运行循环时包含它;要用 TLS,再加上 base_tls.h。
- **base_platform.h**。要使用 platform/ 为自己准备的那些系统调用时包含它, 如 mqtt_platform_random_bytes, 以及用 mqtt_platform_store_open 打开存储区时。
- **base_store.h**。用于存储区由什么构成:持久程度、记录,以及存储区损坏时恢复所报告的内容。base_server.h 和 base_client.h 都包含它。
- **某个版本的头文件**。用于程序要与之比较的常量,如 v5_rc.h 里的原因码。

  程序要用构建库时的页大小 MQTT_PAGE_SIZE 编译,如《构建 libterae》所述:头文件

source-structure-zh.tex Page 9 November 8, 2026 13:46

用它来确定程序交给库的数组的大小。

3

协议代码

src/ 的 36 个源文件就是协议：报文、主题、它们所在的内存、服务器和客户端、让它们的会话状态在重启后仍在的存储区，以及库的版本。它们都在同一个文件夹里，src/port/ 与它们并列（第 4 章）。

3.1 对每个文件都成立的

- **C99，不依赖外部。**每个文件都按 C99 的独立环境（freestanding）编译，并把编译器的警告当作错误。它不做系统调用，不调用分配器，不用 C 库：复制和比较是 base_string.c 自己的。
- **内存来自程序。**库所用的每个数组都由程序在开始时交给它，放在 mqtt_server_storage 或 mqtt_client_storage 里，或作为参数传入。任何资源耗尽都是一种正常的结果，按协议允许的方式报告。
- **没有自己的状态。**没有哪个文件含有它会写入的静态存储期变量。所有状态都在程序拥有并传入的结构里。
- **一个线程，时间作为参数。**没有任何地方加锁、阻塞或等待；需要时间的调用，由调用者以毫秒为单位传入时间。
- **MISRA C:2012。**src/ 和 include/ 按它编写，MISRA 检查器在其中没有发现任何问题。但这并不构成合规声明。

3.2 按层排列的文件

表 3.1 按图 1.1 的层自下而上列出文件。每个文件的头文件与它同名，只是把 .c 换成 .h；服务器和客户端的文件除外，它们的头文件写在各自的组行里。

source-structure-zh.tex Page 12 November 8, 2026 13:46

表 3.1 `src/` 的文件

文件	作用
库	
`base_library.c`	库的版本, 以及构建它时的页大小, 供程序检查它运行时用的共享库。
内存	
`base_string.c`	复制、比较、擦除和测量字节, 不用 C 库。
`base_bool.c`	一些标记, 不必清除内存, 就能区分已写入的内存和只含原有内容的内存。
`base_page.c`	页, 服务器内存被切成的单位, 以及由整页组成的段。
`base_slab.c`	同一大小的槽, 按页分组。
`base_page_arena.c`	从页里取出的任意大小的块: 服务器中大小可变的数据。
`base_arena.c`	在程序给的字节上工作的伙伴分配器: 客户端里大小可变的数据。
`base_pool.c`	同一大小的块组成的池, 由所有连接的环共用。
`base_ring.c`	池里的块组成的一条链, 装着一个连接上等待的字节。
编码	
`base_wire.c`	读写各种数据表示: 整数、可变字节整数、字符串和二进制数据。
`base_utf8_check.c`	检查 UTF-8 编码字符串的字符。
`base_topic_check.c`	检查并匹配主题名和主题过滤器。
`v5_properties.c`	解码和编码 MQTT 5.0 的属性。
`base_packet.c`	解码和编码三个版本的控制报文。
`base_websocket.c`	把 MQTT 放在 WebSocket 上传输: 起始握手和帧。
`base_store.c`	所保存的会话状态的格式: 头部、记录及对它们的检查, 以及读取存储区的一个位置。
主题树和等待分级	
`base_tree.c`	由主题的各个层级组成的树, 存放订阅和保留消息。
`base_tier.c`	把套接字按忙碌程度分级, 使每次等待只需查看少数几个。

续页

source-structure-zh.tex Page 13 November 8, 2026 13:46

表 3.1 src/ 的文件(续)

文件	作用
服务器接口:base_server.h,各文件之间共用 base_server_internal.h	
base_server.c	核心:初始化,读取报文并判断其含义,以及计时器。
base_server_receive.c	收进每个连接发来的字节,把它们切分成报文。
base_server_transmit.c	一个连接接下来发什么,以及什么已经发出去了。
base_server_wait.c	每个连接有多忙,供网络循环决定怎样等待它。
base_server_pool.c	每个连接可以取池里的多少块,以及唤醒等这些块的连接。
base_server_connect.c	CONNECT 和 AUTH,以及接受一个连接。
base_server_publish.c	客户端发来的 PUBLISH,以及确认它的报文。
base_server_route.c	把消息放在内存中,找出它们的去向,并把它们编成字节。
base_server_session.c	会话、订阅和保留消息。
base_server_store.c	重启后仍在的东西的日志,以及等待它的应答。
base_server_snapshot.c	重启后仍在的东西的快照,日志借它重新开始。
base_server_restore.c	放回存储区保存的东西:读取并检查每个位置,选出要应用的部分。
base_server_restore_apply.c	存储区的每条记录对正在恢复的服务器做什么。
客户端接口:base_client.h,各文件之间共用 base_client_internal.h	
base_client.c	程序所调用的东西、为服务器暂存的消息,以及等着发出去的字节。
base_client_receive.c	收进到达的字节,处理服务器发来的报文。
base_client_subscribe.c	SUBSCRIBE 和 UNSUBSCRIBE:检查主题过滤器,取一个报文标识符,并暂存这个报文,直到收到它的应答。
base_client_store.c	它的会话状态的日志和快照,以及等待它们的应答。
base_client_restore.c	放回存储区保存的东西。

source-structure-zh.tex Page 14 November 8, 2026 13:46

3.3 服务器和客户端分在几个文件里

服务器接口是一个头文件 base_server.h，由十三个源文件实现。这些文件共用的东西、它们彼此调用的函数，以及服务器结构的布局，都在 base_server_internal.h 里，程序从不包含它。客户端接口同样是 base_client.h，由五个文件实现，它们共用 base_client_internal.h。两者的存储区和 base_store.c 还共用 base_store_internal.h：一条记录怎样拼成，以及恢复怎样选出它要应用的快照和日志。

接口的一次调用从一个文件开始，可能再经过其它文件。例如，连接发来的字节由 base_server_receive.c 里的 mqtt_server_receive_commit 切分成报文，再由 base_server.c 判断每个报文是什么：CONNECT 接着交给 base_server_connect.c，PUBLISH 交给 base_server_publish.c，其中的消息再交给 base_server_route.c，由它在主题树里找出消息的订阅者。

4

内存函数

编译器即使在没有 C 库的构建里也可能调用 memcpy、memmove、memset 和 memcmp，用来复制一个结构或初始化一个数组。src/port/ 提供它们，供没有 C 库来提供这些函数的程序使用。有 C 库时，这里的东西一个都不编译。

4.1 各文件夹

每种处理器一个文件夹，用取自其它项目的汇编；还有一个 generic/，用 C 写，适用于任何处理器。make PORT=*folder* 把其中一个编译进 libterae_port.a，程序把它紧接在 libterae.a 之后链接；PORT=auto 让编译器来选（第 4.3 节）。

表 4.1 src/port/ 的各文件夹

文件夹	处理器	取自
aarch64	AArch64	Arm Optimized Routines
arm	32 位 Cortex-A, 小端	Arm Optimized Routines、bionic
armv4t	ARM7TDMI、ARM9、ARM11、Cortex-R、大端 Cortex-A	musl、newlib
armv7m	Cortex-M3、M4、M7、M33	Apache NuttX、musl
avr	AVR	avr-libc
i386	x86	musl、newlib
m68k	68000 和 ColdFire	picolibc
mips	MIPS32, o32 ABI	newlib

续页

source-structure-zh.tex Page 16 November 8, 2026 13:46

表 4.1 src/port/ 的各文件夹(续)

文件夹	处理器	取自
powerpc64	64 位 PowerPC, ELFv2 ABI	FreeBSD、NetBSD
riscv32	32 位 RISC-V	newlib
riscv64	64 位 RISC-V, 带 M 扩展	FreeBSD、newlib
rx	RX	picolibc
sh	SH-2 及以后的 SuperH	NetBSD
x86_64	x86-64	FreeBSD
xtensa	Xtensa	newlib
generic	其它任何处理器, 如 Cortex-M0、MSP430 或 8051	libterae 自己的, 用 C 写

sh、rx 和 m68k 已经汇编过,xtensa 已经构建过,但这四个都还没有在各自的处理器上运行过。

4.2 一个文件夹里有什么

- **NOTICE**。这个文件夹里每个文件从哪里来、哪个版本、按什么许可证,以及哪些文件是 libterae 自己的。
- **来自其它项目的文件**。保留原来的名字,如 memcpy.S,逐字节复制,从不编辑。它们包含自己项目的某个头文件时,文件夹里有一个为 libterae 写的同名替代文件,或者把替代文件放在它们所包含的路径上,如 machine/asm.h 或 sys/asm.h。
- **port_*.S 和 port_*.c**。libterae 自己的,也是唯一被编译的文件。每个文件包含来自其它项目的一个文件,并给其中的符号改名;对于这个文件夹没有汇编的函数,则包含 generic/ 里的 C 版本。
- **port.mk**。只在 armv7m/ 里:它的汇编对汇编器的要求。

generic/ 只有它的 NOTICE 和用 C 写的四个函数,它们一次处理一个字节,所以不依赖处理器的任何特性。它的 NOTICE 还列出了一些处理器:没有为它们找到许可证允许 libterae 随库发布的汇编。

这些文件不受库的 C 代码所遵循的规则约束:它们保持各自项目写下的原样。它们所带的每一份许可证,都允许库以它的两种许可证发布。

source-structure-zh.tex Page 17 November 8, 2026 13:46

4.3 PORT=auto 怎样选择

src/port/select.in 不是 C，也没有任何东西包含它。用 PORT=auto 时，Makefile 用这次构建的编译选项让编译器的预处理器处理它，处理后只留下一个文件夹的名字：按编译器所定义的宏进行测试，第一个与处理器相符的文件夹。一个都不相符时得到 generic；不生成 ELF 目标文件的编译器，如 macOS 和 Windows 上的，也得到 generic，因为每个汇编文件夹都是为 ELF 写的。

用汇编文件夹时，base_string.c 用那个文件夹的 memcpy 复制，而不用它自己的循环，于是库就需要 libterae_port.a。

5

平台

src/ 中因每个系统做法不同而略去的部分，都在 platform/ 里，它是库针对某个系统的特化。它有套接字和等待套接字、时钟、随机字节、程序的输出和停止、TLS，以及让会话状态在重启后仍在的存储区，还有服务器和客户端的网络循环。它是库里唯一调用系统的代码。程序可以用 OS=none 把它省去，把 src/ 接到自己的循环上。

5.1 两个层次

图 5.1 把这个文件夹分两个层次画出。

- **platform/ 的顶层。**只依据系统的文件夹所实现的头文件编写，所以在每个系统上都一样：base_platform.h、platform_wait.h，以及存储区所用的 platform_store.h（文件）或 platform_flash.h（闪存）。它不包含任何系统的头文件：只包含 C 的标准头文件，platform_tls.c 里还包含 Mbed TLS 的头文件。
- **每个系统一个文件夹。**那个系统按自己的方式做的事，只依据该系统的文档所说明的东西编写，不用别的。每个文件夹为自己的系统实现 base_platform.h，以及它的构建所需的顶层头文件。

source-structure-zh.tex Page 20 November 8, 2026 13:46

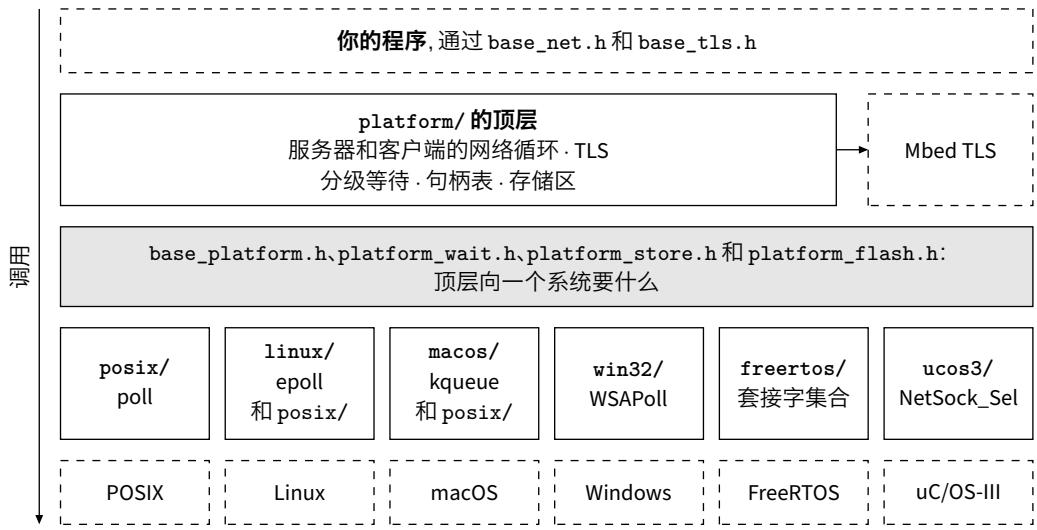


图 5.1 platform/, 从程序到系统。如图 1.1 所示, 顶层还调用服务器接口、客户端接口、等待分级与 WebSocket

5.2 顶层的文件

表 5.1 platform/ 顶层的文件

文件	作用	编译用于
网络循环:base_net.h		
platform_net_server.c	在一个线程上、在真实连接上运行一个服务器: 等待就绪的套接字, 读取它们, 并经 TCP、TLS 或 WebSocket 把服务器为每个连接准备好的内容写出去。	每个系统
platform_net_client.c	在一个连接上运行一个客户端。	每个系统
platform_net_runs.c	在套接字上或经 TLS 写出字节段, 供两者共用。	每个系统
platform_net_store.c	在这些循环中运行的服务器或客户端的存储区: 恢复它, 封好并同步每一轮的记录, 写出快照。	每个系统
platform_net.h	各循环和它们的存储区共用的东西。	–

续页

source-structure-zh.tex Page 21 November 8, 2026 13:46

表 5.1 platform/ 顶层的文件(续)

文件	作用	编译用于
TLS:base_tls.h		
platform_tls.c	通过 Mbed TLS 实现 TLS, 其内存取自一个静态池。不带 TLS 构建时, 每个函数都报告 MQTT_TLS_ERROR。	每个系统
mbedtls_config.h	libterae 在 Mbed TLS 的配置上添加的内容。Mbed TLS 要用它来构建。	Mbed TLS
分级等待:base_platform.h		
platform_wait.c	在接受一个条目数组(每个套接字一项)的系统上分级等待, 如 poll。	posix、win32、ucos3
platform_wait_set.c	在自己维护套接字集合、并报告其中哪些已就绪的系统上等待, 如 epoll 和 kqueue。	linux、macos、freertos
platform_wait.h	两者需要系统的文件夹提供的东西。	-
存储区:base_platform.h		
platform_store_file.c	把存储区保存在一个文件夹的文件中: 快照的各份副本和两个日志文件, 移到一旁时放进自己的文件夹。	posix、linux、macos、win32
platform_store.h	它需要系统的文件夹提供的文件操作。	-
platform_store_flash.c	把存储区保存在板子指定的一块闪存区域中, 这块区域被分成由整个擦除块组成的位置。	freertos、ucos3
platform_flash.h	它需要系统的文件夹提供的闪存操作, 由板子给出。	-
base_platform.h 的其余部分		
platform_slots.c、platform_slots.h	一张句柄表, 句柄从空闲链表中分配, 用于套接字不是小整数的系统。	win32、freertos
platform_parse.c	从文本里读数字和持久程度的名字, 不需要系统提供任何东西。	每个系统

续页

source-structure-zh.tex Page 22 November 8, 2026 13:46

表 5.1 platform/ 顶层的文件（续）

文件	作用	编译用于
platform_probe_none.c	base_platform.h 为测试和基准测试保留的那些调用，用于没有可复制的进程、也没有信号的系统：每个调用都报告自己做不到。	win32、freertos、ucos3

5.3 各系统的文件夹

各文件夹的文件承担相同的角色，在文件夹名之后接同一个词来命名：_socket.c 用于套接字；_system.c 用于时钟、睡眠、程序的输出和停止，posix/ 把它分成 posix_clock.c 和 posix_signal.c；_random.c 用于随机字节；还有一个用于等待的文件，它给 platform_wait.c 提供条目，或维护 platform_wait_set.c 的集合。存储区在文件中时用 _file.c 和 _sync.c，在闪存中时用 _flash.c。一个文件夹的 _internal.h 放的是该文件夹各文件共用的东西。

表 5.2 各系统的文件夹

文件夹	用什么	一次构建取什么
posix	只用 POSIX.1-2008，以及 poll，有两个例外：随机字节来自 /dev/urandom，一个 POSIX 没有提到的文件；fsync 来自 POSIX 可选的文件同步（File Synchronization）。	它的十个文件：posix_socket.c、 posix_options.c、posix_signal.c、 posix_clock.c、posix_wait_one.c、 posix_probe.c、posix_file.c、 posix_poll.c、posix_random.c、posix_sync.c；以及 platform_wait.c、platform_store_file.c
linux	posix/，等待用 epoll，随机字节用 getrandom。	linux_epoll.c、linux_random.c；posix/ 中除 posix_poll.c 和 posix_random.c 以外的文件；以及 platform_wait_set.c、platform_store_file.c

续页

source-structure-zh.tex Page 23 November 8, 2026 13:46

表 5.2 各系统的文件夹(续)

文件夹	用什么	一次构建取什么
macos	posix/，等待用 kqueue，随机字节用 getentropy，同步文件用 F_FULLFSYNC，因为那里的 fsync 到不了磁盘自己的缓存。	macos_kqueue.c、 macos_random.c、macos_sync.c；posix/ 中除 posix_poll.c、posix_random.c 和 posix_sync.c 以外的文件；以及 platform_wait_set.c、platform_store_file.c
win32	Winsock 2 和 Windows API，32 位和 64 位，用 WSAPoll 等待。随机字节来自 BCryptGenRandom。	win32_socket.c、 win32_system.c、win32_poll.c、 win32_random.c、win32_file.c；以及 platform_wait.c、platform_slots.c、platform_probe_none.c、platform_store_file.c
freertos	内核和 FreeRTOS+TCP，用它的套接字集合等待。随机字节来自程序，经 xApplication GetRandomNumber；存储区的闪存来自板子，经 freertos_board_store_ 系列调用。	freertos_socket.c、freertos_system.c、freertos_select.c、freertos_random.c、freertos_flash.c；以及 platform_wait_set.c、platform_slots.c、platform_probe_none.c、platform_store_flash.c
ucos3	uC/OS-III 和 uC/TCP-IP，用 NetSock_Sel 等待。随机字节来自程序所用的板子，经 ucos3_board_random；存储区的闪存经 ucos3_board_store_ 系列调用。	ucos3_socket.c、 ucos3_system.c、ucos3_select.c、 ucos3_random.c、ucos3_flash.c；以及 platform_wait.c、platform_probe_none.c、platform_store_flash.c

一个系统的每次构建，还取表 5.1 中标为“每个系统”的那六个文件。

5.4 Makefile 挑选文件

platform/ 里没有哪个文件去判断自己在哪个系统上。make OS=*folder* 取表 5.2 里那个文件夹的文件，每个文件都用其所在文件夹的定义编译：platform/ 的顶层不用任何定义，posix/、linux/ 和 macos/ 用 _POSIX_C_SOURCE，win32/ 用它所需要的 Windows 版本，

source-structure-zh.tex Page 24 November 8, 2026 13:46

实时系统用程序在 RTOS_INCLUDES 里指定的头文件。不指定 OS 时，构建取它所在机器对应的文件夹。OS=none 不取 platform/ 的任何东西，也就没有 TLS。《构建 libterae》逐项讲了这些。

一个新的系统就是一个新的文件夹：它实现 base_platform.h，按这个系统等待套接字的方式选用 platform_wait.c 或 platform_wait_set.c，按它把存储区放在哪里选用 platform_store_file.c 或 platform_store_flash.c，并为之实现 platform_store.h 或 platform_flash.h，还在 Makefile 里有自己的文件清单。

6

去哪里找

表 6.1 为库的每一部分列出首先该打开的文件。文件开头的那段注释会再说明它与哪些其它文件协同工作。

表 6.1 库的每一部分在哪里

要看	打开
使用库	
实现一个服务器, 以及交给它的内存	`include/base_server.h`
实现一个客户端	`include/base_client.h`
让 `platform/` 运行循环	`include/base_net.h`
让会话状态在重启后仍在	`include/base_store.h`
一次调用的结果	`include/base_status.h`
版本, 以及共享库是否适合程序	`include/base_library.h`

原因码和返回码	`include/v5_rc.h`
	`include/v311_rc.h`
	`include/v31_rc.h`

服务器	
对报文的检查, 以及报文的含义	`src/base_server.c`
把字节切成报文	`src/base_server_receive.c`
CONNECT、AUTH 和分配的客户端标识符	`src/base_server_connect.c`

一条消息, 从 PUBLISH 到它的订阅者	`src/base_server_publish.c`
	`src/base_server_route.c`
	`src/base_tree.c`

续页

source-structure-zh.tex Page 26 November 8, 2026 13:46

表 6.1 库的每一部分在哪里（续）

要看	打开
会话和保留消息	`src/base_server_session.c`

传输中的字节，以及每个连接能持有多少	`src/base_pool.c`
	`src/base_ring.c`
	`src/base_server_pool.c`

它的存储区：日志、快照和恢复	`src/base_server_store.c`
	`src/base_server_snapshot.c`
	`src/base_server_restore.c`

客户端	
连接、发布，以及暂存的消息	`src/base_client.c`
服务器发来的内容	`src/base_client_receive.c`
订阅	`src/base_client_subscribe.c`
它的存储区	`src/base_client_store.c` `src/base_client_restore.c`
协议的字节	
控制报文	`src/base_packet.c`
整数、字符串和二进制数据	`src/base_wire.c`
UTF-8、主题名和主题过滤器	`src/base_utf8_check.c` `src/base_topic_check.c`
MQTT 5.0 的属性	`src/v5_properties.c`
WebSocket	`src/base_websocket.c`
存储区的记录，以及对它们的检查	`src/base_store.c`
一个系统	
服务器的网络循环	`platform/platform_net_server.c`

等待套接字	`src/base_tier.c`
	`platform/platform_wait.c`
	`platform/platform_wait_set.c`

某个系统的套接字	`platform/`*`os`*`/`，以 `_socket.c` 结尾的那个文件

续页

source-structure-zh.tex Page 27 November 8, 2026 13:46

表 6.1 库的每一部分在哪里(续)

要看	打开
TLS	`platform/platform_tls.c` `platform/mbedtls_config.h`

文件或闪存中的存储区	`platform/platform_net_store.c`
	`platform/platform_store_file.c`
	`platform/platform_store_flash.c`

某个处理器的内存函数	`src/port/`*folder*`/NOTICE`